

SOEN 6481

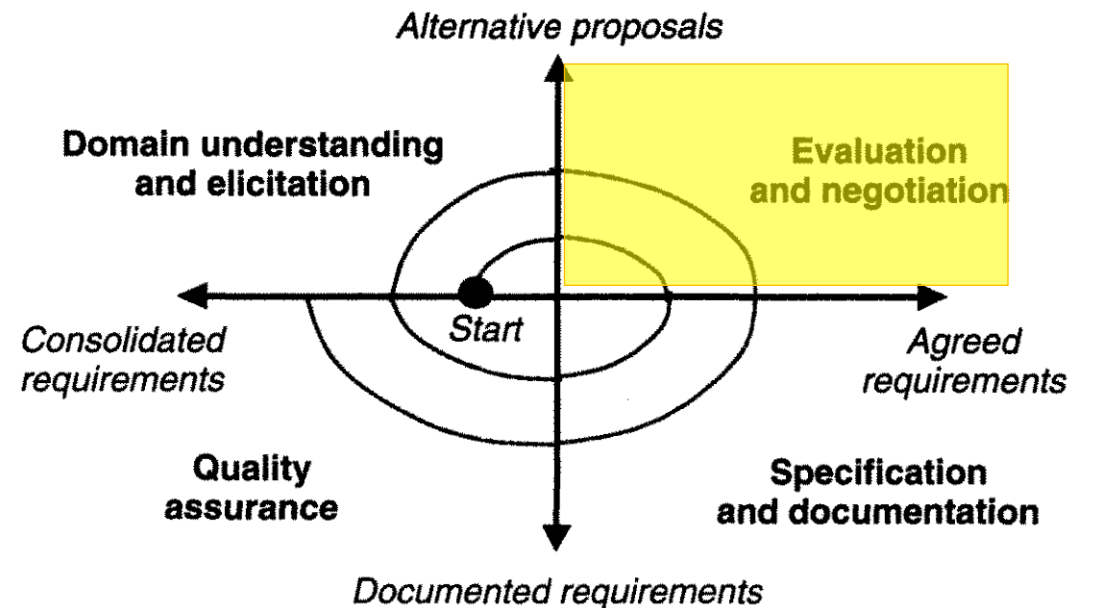
P1. Requirements Evaluation and Risk analysis

Inconsistency mgmt. learning objectives

- Understand the role of inconsistency management
- Learn about different types of inconsistencies
- Learn a structured conflict management process
- Learn different techniques for identifying overlaps and conflicts
- Learn how to document overlaps and conflicts with an interaction matrix
- Learn different techniques for generating conflict resolutions

Introduction

- The techniques discussed in requirements elicitation, aim to identify stakeholder needs together with alternative ways of addressing these in the system-to-be.
- Following the spiral model of the RE process, we now need to evaluate the elicited requirements and assumptions on several grounds.



Reasons to evaluate the elicited req.

4

Requirements
can be
inconsistent
with each other,
due to their
origin

Requirements
can be
overexposed to
risks

Alternatives
must be
compared to
select the best
options for our
system

Requirements
must be
prioritized in
the selected
alternatives

Inconsistency management

- Inconsistencies are typically the rule, not the exception.
- They arise from multiple viewpoints and concerns.
- Inconsistencies must be detected and eventually resolved.
- Inconsistency management deals with principles and techniques for handling them.
- Conflicts are important type of inconsistency, which deserves **special attention**.



Types of inconsistency

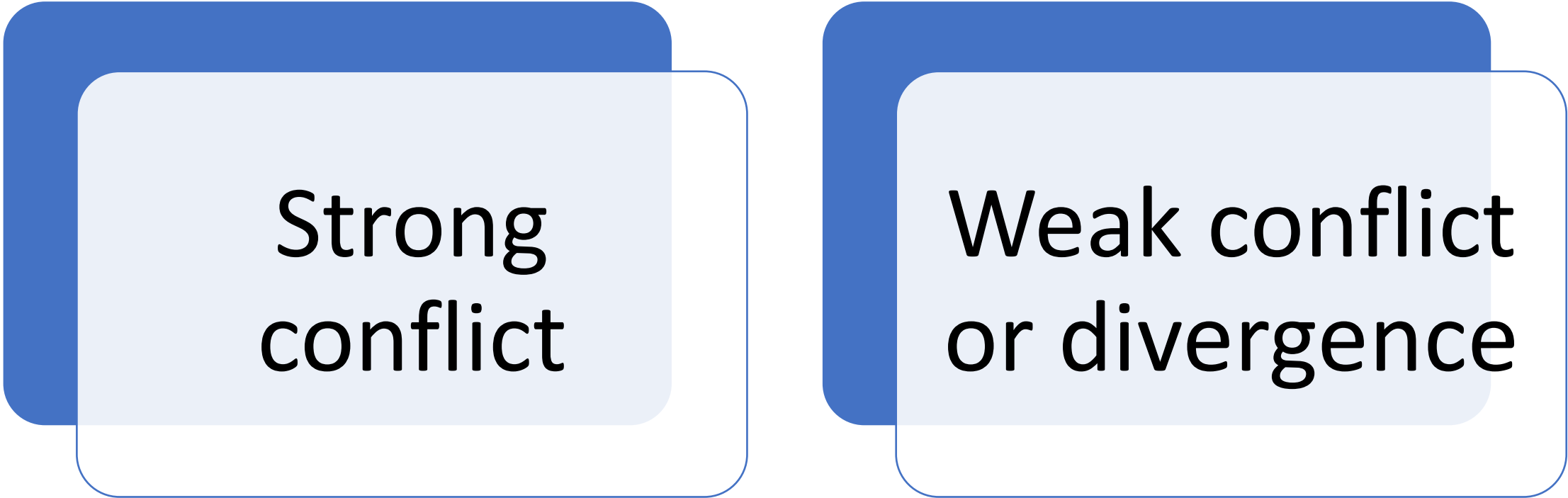


Terminology
clash

Designation
clash

Structure
clash

Types of inconsistency (contd.)



**Strong
conflict**

**Weak conflict
or divergence**

Handling inconsistencies (terminology clash)

- Clashes in terminology, designation and structure are most effectively countered by careful elaboration of a **glossary of terms** that everyone agrees and sticks to it.
- Such a glossary should provide precise, intelligible definition of all terms used and if necessary, a list of accepted synonyms.
- **Glossary elaboration takes place during domain understanding and requirements elicitation.**

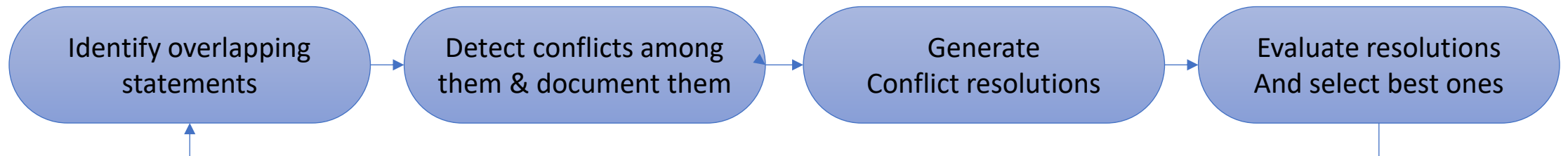
Handling inconsistencies (strong/weak conflict)

- Handling strong and weak conflicts is more difficult.
- Conflicts are generally rooted in two kinds of problems:
 1. Stakeholders have different objectives and priorities
 2. There are inherent incompatibilities between non-functional requirements, or between functional and non-functional requirements

**Negotiation
required**

Managing conflicts: a systematic process

- Robinson et. al., proposed a more systematic process through a few steps with dedicated techniques for each step.



Identify overlapping statements

- Statements overlap each other if they refer to some common or inter-related phenomena.
- Example: the phenomena of **acquiring**, **borrowing** and **returning** book copies are inter-related as they refer to the common concept of 'book copy'.



Detecting and documenting conflicts

Informal detection

- Are overlapping statements satisfiable together and when not?

Heuristic detection (Lamsweerde et al.)

- Check *information requirements* vs *confidentiality requirements* that refer to related objects
- Check requirements on *decreasing* some quantity vs requirements on *increasing* a related quantity

Detecting and documenting conflicts (contd.)

- Formal detection
 - Formal techniques: based on inconsistency checking, theorem proving or derivation of boundary conditions.
 - Requires overlapping statements to be formalized in a **logic-based** specification language.
- Lightweight detection
 - We may check if overlapping statements match some assertion templates that were proved once for all to be conflicting with each other (conflict patterns).

Documenting conflicts

- Once they have been detected, conflicts should be documented for later resolution
- A standard documentation technique consists of building an interaction matrix (Kotonya & Sommerville, 1997)

Statement	S1	S2	S3	S4	Total
S1	0	1000	1	1	1002
S2	1000	0	0	0	1000
S3	1	0	0	1	2
S4	1	0	1	0	2
Total	1002	1000	2	2	2006

Documenting conflicts

1 If statement S_i conflicts with statement S_j . Otherwise
0

Statement	S1	S2	S3	S4	Total
S1	0	1000	1	1	1002
S2	1000	0	0	0	1000
S3	1	0	0	1	2
S4	1	0	1	0	2
Total	1002	1000	2	2	2006

Documenting conflicts

1000 if
statement S_i
overlaps
without
conflicting
with statement
 S_j . Otherwise
0

Statement	S1	S2	S3	S4	Total
S1	0	1000	1	1	1002
S2	1000	0	0	0	1000
S3	1	0	0	1	2
S4	1	0	1	0	2
Total	1002	1000	2	2	2006

Documenting conflicts

- The total number of non-conflicting overlaps and conflicts is given by the quotient and remainder of the integer division of **Total | Total** by 1000
- Example $2006/1000 = 2.006$

Statement	S1	S2	S3	S4	Total
S1	0	1000	1	1	1002
S2	1000	0	0	0	1000
S3	1	0	0	1	2
S4	1	0	1	0	2
Total	1002	1000	2	2	2006

 Non-conflicting overlap

 Conflict

Generating conflict resolutions

- We must resolve detected conflicts:
 - Before software development starts
 - Not too soon, to allow for further elicitation of useful information within individual viewpoints
- **Using elicitation techniques**
 - Use the techniques reviewed in Lecture 2.p2, to elicit alternative conflict resolutions with stakeholders
 - Example: interviews, group sessions.
 - The target resolution should capture a reasonable compromise for all parties involved.



Generating conflict resolutions with tactics

- We may produce resolutions systematically by use of *operators* that encode conflict-resolution tactics.
 - Some operators transform the conflicting statements or objects involved in such statements. Other operators introduce new requirements.
1. Avoid boundary condition
 - Ensure that the boundary condition for a conflict can never become true
 - 'A borrower should return a borrowed book copy within two weeks' and 'A borrower should keep a borrowed book copy as long as he or she needs it'.
 - Avoiding this boundary condition might be achieved by keeping some copies of popular books always unable to be borrowed; such copies are available for direct use in the library at any time when needed.

Generating conflict resolutions with tactics

2. Restore conflicting statements

- Ensure that the conflicting statements become together satisfiable again soon after the boundary condition has occurred. Forcing borrowers to return book copies even if they are still needed.

3. Weaken conflicting statements

- Make one or several of the conflicting statements less restrictive so that the conflict no longer exists
- 'A borrower should return a borrowed book copy within two weeks **unless he or she gets explicit permission to keep it longer for some good reason**'

4. Drop lower-priority statements

- This is an extreme case of the previous tactic where one or several lower-priority statements involved in the conflict are weakened to the point that they are universally true. E.g., **R1 (High Priority):** All users must authenticate using multi-factor authentication (MFA). **R2 (Low Priority):** Users should be able to log in within **2 seconds**.
- Then, we change 'Users should be able to log in within **2 seconds**' to 'The system should **allow users to log in**'.

Generating conflict resolutions with tactics

5. Specialize conflict source or target

- Identify the source (or target) objects involved in conflicting statements and specialize these so that the conflict disappears
- Example: the conflict between the statements 'Allow users to be informed about the loan status of books' and 'Do not allow students to know which user has borrowed what'.
- Specializing the conflict source object 'user' into 'staff user' so that the first statement is transformed into the conflict-free version **'Allow staff users to be informed about the loan status of books'**
- Alternatively, the conflict target object 'loan' can be specialized into an **anonymized version** in which status information no longer covers the identity of borrowers

Evaluating conflict resolutions

- Once alternative conflict resolutions have been explored, we need to decide on which 'best' resolution to select.
- The basis for this decision is provided by evaluation criteria such as the degree of contribution of the resolutions to critical nonfunctional requirements, and the reduction of other risks and conflicts.
- Techniques for this will be discussed later.



Risk analysis



Learning objectives

- Understand the role of risk management in requirements evaluation.
- Learn about different types of risks.
- Learn different risk identification techniques, including risk checklists, component inspection, and risk trees.
- Learn qualitative and quantitative risk assessment techniques.
- Understand how to find risk countermeasures.
- Learn how to select appropriate countermeasures.
- Understand NASA's Defect Detection Prevention (DDP) approach.

Introduction

- Many software failures originate from our natural inclination to conceive over-ideal systems.
- However, moving from the *system-as-is* to the *system-to-be* inevitably raises several types of risk.
- **If risks go unrecognized or underestimated, the requirements will be incomplete or inadequate.**



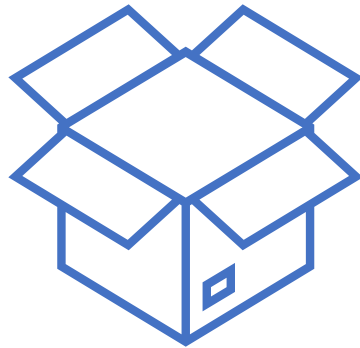
Risk definition

- A risk is an uncertain factor whose occurrence may result in a *loss of satisfaction* of a corresponding objective.
- It has a *likelihood* of occurrence and one or several undesirable consequences associated with it.
- Each consequence has a *severity* in terms of degree of *loss of satisfaction* of the corresponding objective.

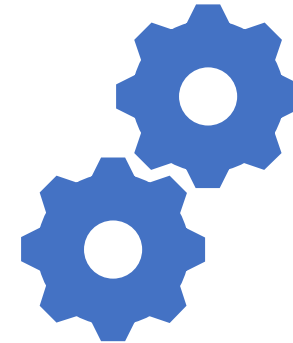


Types of risks

As risk typically poses negative impact on corresponding objectives, risk types correspond to types of objectives that could be missed in the system-to-be:



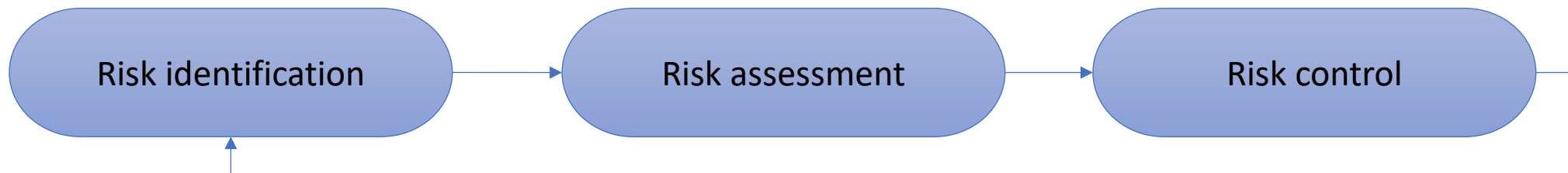
Product-related risk



Process-related risks

Risk management

- It is an essential part of requirements evaluation.
- Some requirements or assumptions might not take risks sufficiently into account.
- When this is the case, we must identify new requirements as *countermeasures* to risks that are **too likely** or whose possible consequences are **too severe**.
- We should then evaluate *countermeasures*, which might involve new risks or conflicts with other important requirements.

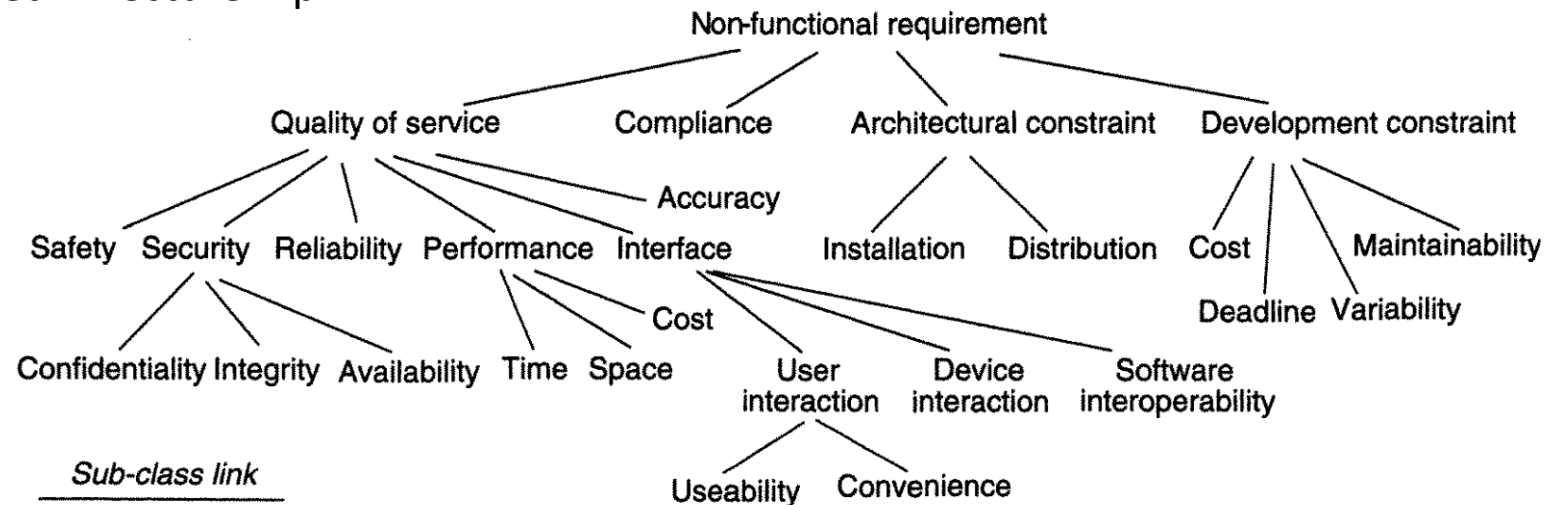


Risk identification

- The following techniques can be used to make awareness of possible risks impacting the objectives of our project

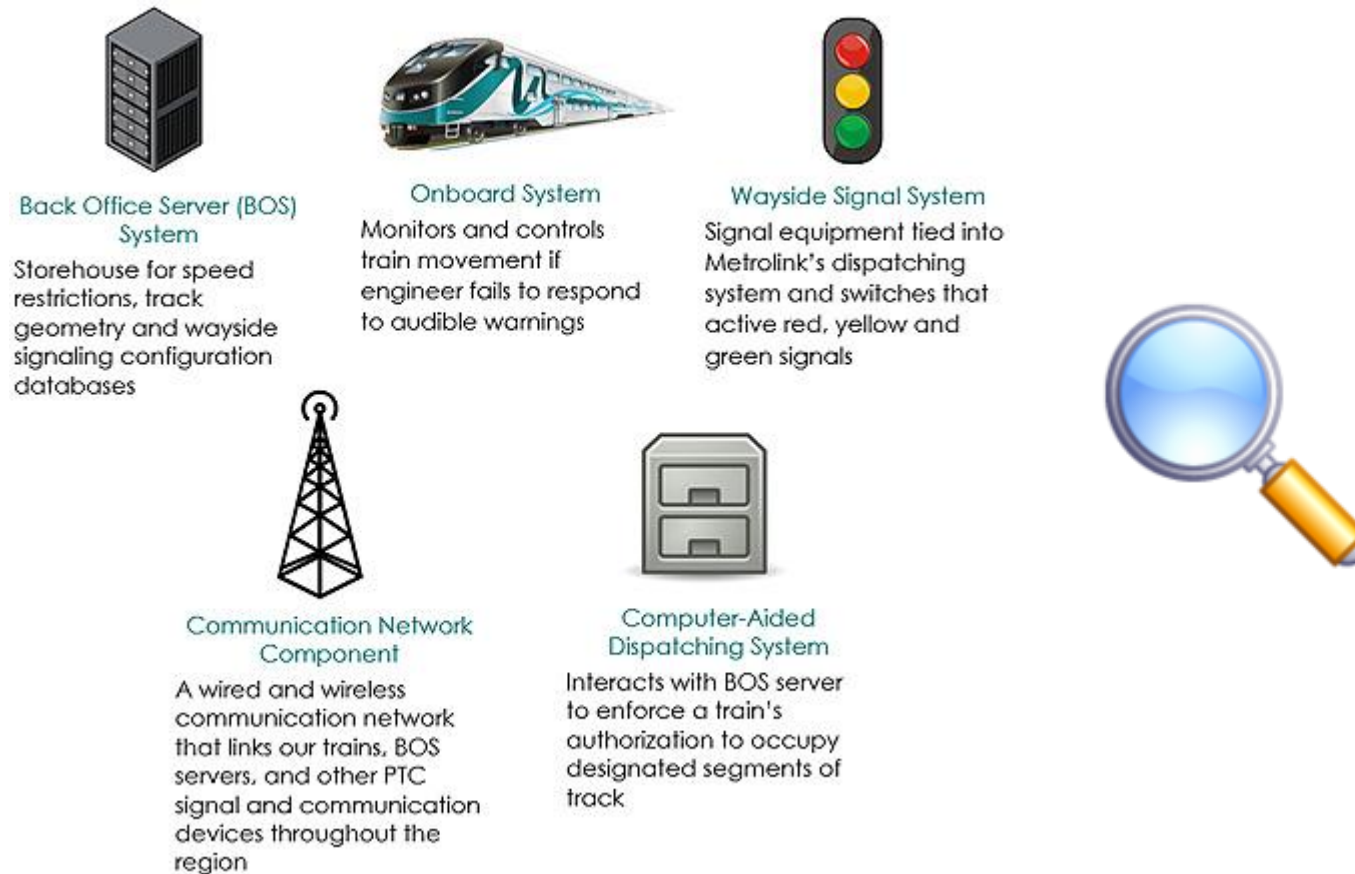
1. Risks checklists

- Checklists can be built from risk categories that negatively impact on corresponding requirements
- Such checklists can be built from risk categories that impact on corresponding requirements categories introduced in lecture 2.p2.



2. Component inspection

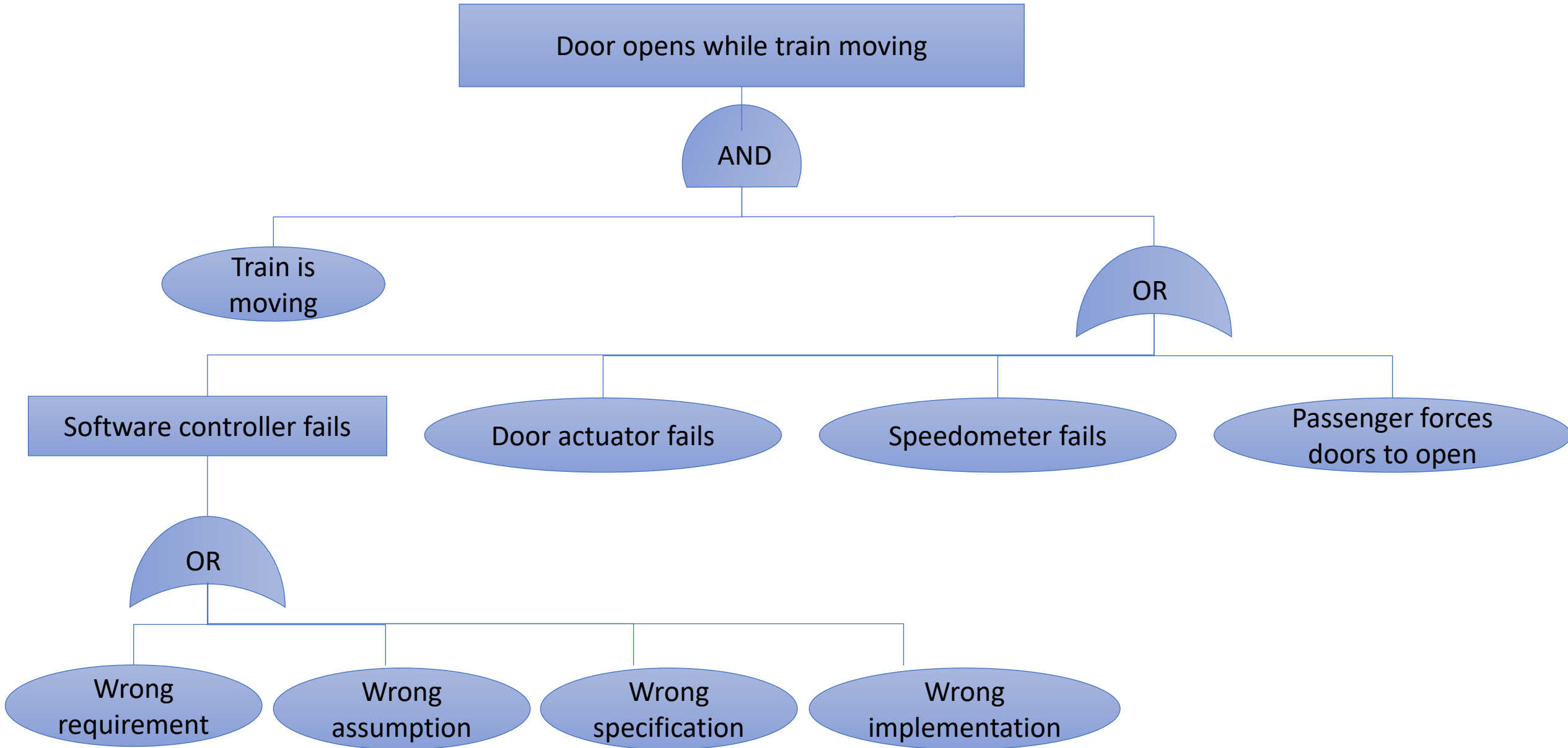
It consists of reviewing the various components



3. Risk trees

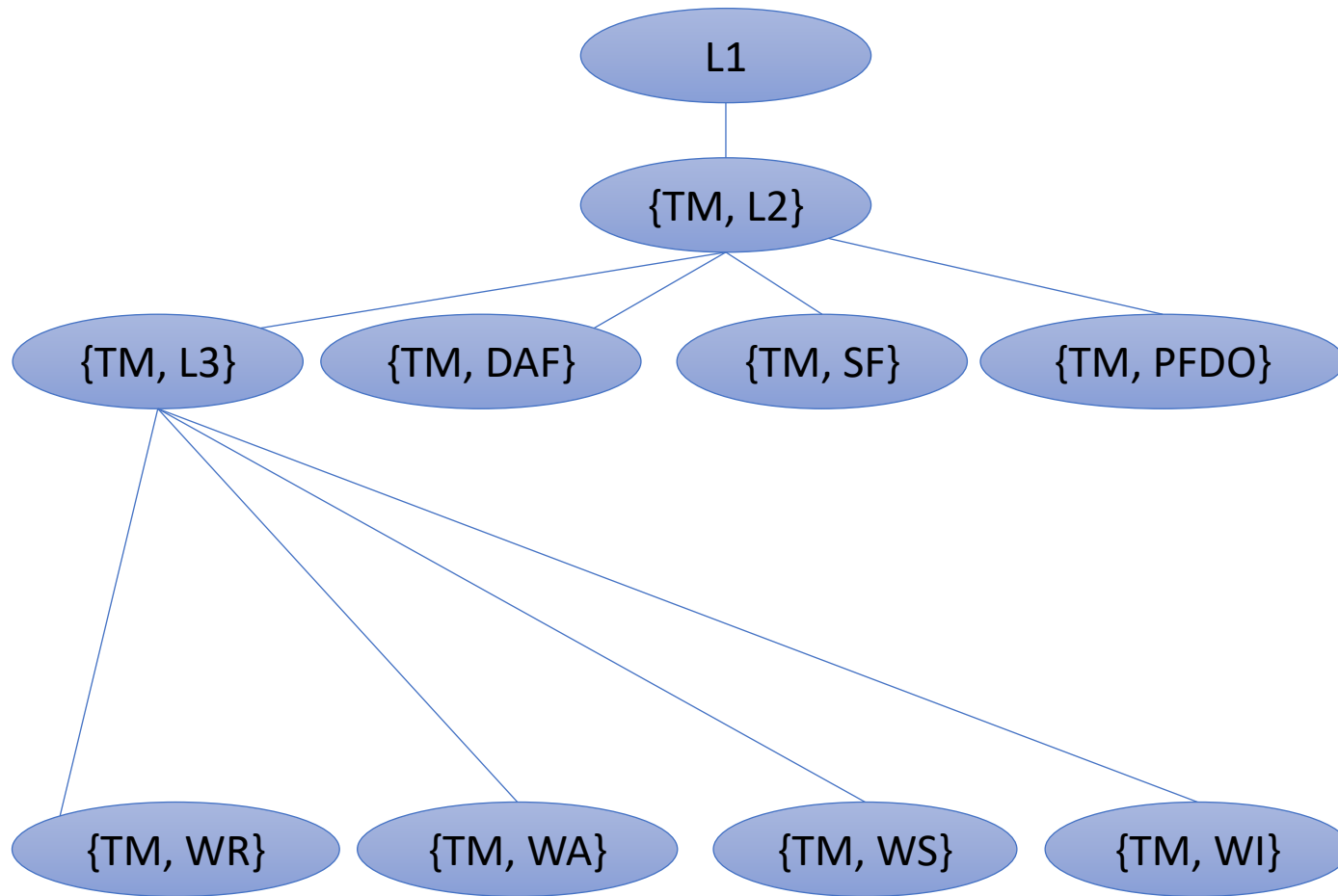
- The identification of risks through component inspection can be made more systematic using risk trees (also called fault trees).
- Risk trees have two kinds of node:
 - Failure nodes capture independent failure events or conditions
 - Logical nodes are AND or OR nodes that capture causal links
- Once a risk tree has been built, we can enumerate all minimal AND combinations of leaf events or conditions, each of which is sufficient for causing the root failure node (the set is called the cut set of the risk tree)

Portion of fault tree for train door control system



Rules to obtain the cut set of the risk tree

1. The top node of the cut-set tree is the top logical node of the risk tree
2. If the current node in the cut-set tree is an *OR* node, it is expanded in as many child nodes as there are alternative child nodes in the risk tree; if it is an *AND* node, it is expanded into one single aggregation node composed of all conjoined child nodes in the risk tree
3. The process terminates when the child nodes obtained are all basic events or conditions or aggregations of basic events or conditions.



The cut-set:

$\{\{TM, WR\}, \{TM, WA\}, \{TM, WS\}, \{TM, WI\}, \{TM, DAF\}, \{TM, SF\}, \{TM, PFDO\}\}$.

4. Using elicitation techniques

The elicitation techniques reviewed in Lecturer 2.p2. can also be used to identify system-specific risks:

- *Scenarios* may be used to raise *WHAT IF* questions and point out failure situations
- *Knowledge reuse* techniques may be applied to risks previously experienced with similar systems or within the same organization.
- *Group sessions* may be specifically dedicated to the identification of project-specific risks.

Risk assessment: qualitative assessment

- The identified risks should be assessed in terms of the likelihood of their occurrence and the severity of their possible consequences
- We must prioritize the risks and determine an appropriate response for likely risks that have severe consequences
- Example: Severity of consequences by risk likelihood levels for “**Doors open while train moving**”

Consequences	Likely	Possible	Unlikely
Loss of life	Catastrophic	Catastrophic	Severe
Serious injuries	Catastrophic	Severe	High
Train car damaged	High	Moderate	Low
No. of passengers decreased	High	High	Low
Airport reputation damaged	Moderate	Low	Low

Risk assessment: quantitative assessment

- Alternatively, we may use numerical scales for risk estimation and comparison

Risk exposure = impact x risk probability

- ▶ For a quantitative model, we calculate the product of impact and probability. The product is referred to as *risk exposure*. Once the risk exposure is calculated, risk items can be ranked.
- ▶ **Example:** Consider a fire which can cause 0.5 millions of damage in a facility. Let the probability of this event be 0.01. Then, the risk exposure is

$$RE = \$0.5m \times 0.01 = \$5,000.$$

which would be analogous to the amount needed to purchase insurance.

Risk Control

- Once we have identified and assessed product- and process-related risks, we must address these.
- High-exposure risks must be reduced through counter measures
- This reduction should be *cost-effective*
- We may identify countermeasures through:
 - Using elicitation techniques (lecture 2.p2.)
 - Reusing known countermeasures
 - Using risk-reduction tactics

Risk reduction tactics

- *Reduce risk likelihood*
 - Introduce new requirements to ensure that the likelihood of occurrence of the risk is reduced.
 - example: to send train drivers prompts for drivers' reactions to acceleration commands.
- *Avoid risk*
 - Introduce new requirements ensuring that this specific risk may never occur.
 - example: software controller checks train's speed before responding to opening requests from passengers.
- *Reduce consequence likelihood*
 - Introduce new requirements ensuring that the likelihood of occurrence of this consequence of the risk is significantly reduced.
 - example: the likelihood of severe injuries or loss of life if the train doors open during movement is reduced by generating an alarm within train cars in the case of door opening when a train is moving

Risk reduction tactics (contd.)

- *Avoid risk consequence*
 - Introduce new requirements prescribing that a severe consequence of this tolerated risk may never occur.
 - For example, new requirements might be introduced to ensure specifically that train collisions cannot occur in case the risk of inaccurate train position or speed information does occur
- *Mitigate risk consequence*
 - Introduce new requirements to reduce the severity of consequences of this tolerated risk
 - For example, consider the risk of important meeting participants having a last-minute impediment
 - The absence of such participants can be mitigated in the system by integrating new facilities such as videoconferencing, appointment of proxies, and the like.

Risk reduction leverage (RRL)

- ▶ We define *risk reduction leverage* as the ratio of the reduction in risk exposure over the cost of the reduction (countermeasure).

$$\text{Risk reduction leverage (RRL)} = \frac{RE_{\text{before}} - RE_{\text{after}}}{\text{Cost of risk reduction}}$$

- ▶ According to the definition, values of *RRL* greater than 1 indicate cost effective risk reduction measures because the reduction in risk exposure achieved by taking some measure is greater than the costs involved.
- ▶ On the other hand, values of *RRL* less than 1 would indicate non cost effective measures.

Risk reduction leverage (RRL) exercise

- ▶ Consider a server which maintains a repository of data. The probability of losing the data is 15%. The cost of losing the data is measured in terms of the cost of its reproduction and re-entry into the database which is estimated at \$30,000.
- ▶ In order to reduce the risk of losing the data, the company requested the services of a consultant who proposed the following two options:
 - ▶ Option 1. Frequent backup.
 - ▶ Option 2. Replication of database to a parallel server.

$$\text{Risk reduction leverage (RRL)} = \frac{RE_{\text{before}} - RE_{\text{after}}}{\text{Cost of risk reduction}}$$

$$RE = \text{risk probability} \times \text{amount at stake}$$

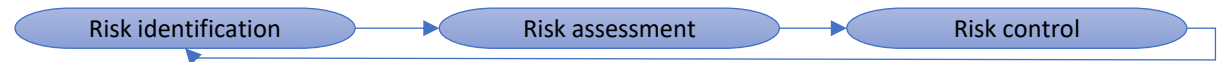
- ▶ The first option is estimated to reduce the risk to 10%. It is estimated to cost \$2000. The second option is estimated to reduce the risk to 5%. It is estimated to cost \$2500.
- ▶ As a project manager you need to make the following decisions: Calculate the Risk Reduction Leverage factor for the two options above.

Which option would you adopt, if any?

$$\begin{aligned} RRL_1 &= 0.75 \\ RRL_2 &= 1.2 \end{aligned}$$

Risk Documentation

- The *Identify-Assess-Control* cycle may need to be repeated as countermeasures can introduce new risks in their turn
- This process should be documented in the requirements document to provide rationale for countermeasure requirements and to support requirements evolution
- Risk documentation should include:
 - The conditions or events characterizing its occurrence
 - The estimated likelihood of occurrence
 - The possible causes and consequences
 - The estimated likelihood and severity of each possible consequence
 - The countermeasures that were identified together with their respective risk-reduction leverage
 - The selected subset of countermeasures



References

1. Chapter 3, Axel van Lamsweerde. Requirements Engineering: From System Goals to UML Models to Software Specifications. John Wiley & Sons, 2009.
2. Fault tree analysis. <https://www.smartdraw.com/fault-tree/>